tarefa_numpy

October 3, 2026

1. criando e analisando um array

[ ]:
```
import numpy as np

x = np.array([2, 4, 6, 8, 10])

print(x.ndim)
print(x.size)
print(x.dtype)
```

```
1
5
int64
```

2. operações vetoriais

[8]:
```
x = np.array([1, 2, 3, 4, 5])

print(2*x + 3)
```

```
[ 5  7  9 11 13]
```

3. criação de uma matriz

[ ]:
```
A = np.arange(1, 17).reshape(4, 4)

print(A)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

4. soma das linhas e colunas

[32]:
```
A = np.arange(1, 10).reshape(3, 3)

print(A)
n, m = np.shape(A)

for i in range(n):
   print(f'soma da linha {i} = {sum(A[i])}')
```

```
for j in range(m):
   print(f'soma da coluna {j} = {sum(A[:, j])}')
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
soma da linha 0 = 6
soma da linha 1 = 15
soma da linha 2 = 24
soma da coluna 0 = 12
soma da coluna 1 = 15
soma da coluna 2 = 18
```

5. produto matricial

[ ]:
```
A = np.array([
    [1, 2], [3, 4]
    ])

B = np.array([
    [5, 6],
    [7, 8]
])

print(A @ B)
print(np.matmul(A, B))
print(np.dot(A, B))
```

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

6. determinante e inversa

[96]:
```
import numpy.linalg as la

A = np.array([
    [4, 7],
    [2, 6]
])

print(f'det A = {la.det(A)}')
print(f'A inversa =\n{la.inv(A)}')
```

```
det A = 10.000000000000002
A inversa =
[[ 0.6 -0.7]
```

```
[-0.2  0.4]]
```

7. sistema linear

[51]:
```
A = np.array([
    [3, 2],
    [1, -1]
])

b = np.array([7, 1])

x = np.linalg.solve(A, b)
print(x)
```

```
[1.8 0.8]
```

8. autovalores

[97]:
```
A = np.array([
    [4, 1],
    [2, 3]
])

print(f'autovalores de A: {la.eigvals(A)}')
```

```
autovalores de A: [5.+0.j 2.+0.j]
```

9. estatística

[55]:
```
notas = np.array([7, 8, 5, 9, 10, 6, 8, 7])

print(f'média: {np.mean(notas)}')
print(f'mediana: {np.median(notas)}')
print(f'variância: {np.var(notas)}')
print(f'desvio padrão: {np.std(notas)}')
```

```
média: 7.5
mediana: 7.5
variância: 2.25
desvio padrão: 1.5
```

10. seleção por condição

[98]:
```
x = np.array([3, 12, 7, 20, 5, 18, 2, 25])

print(x[x > 10])
```

```
[12 20 18 25]
```

11. aproximação de uma integral

[70]:
```
x = np.linspace(0, np.pi, 101)
y = np.sin(x)

I = np.trapezoid(y, x)

print(f'integral aproximada: {I:.8f}')
print(f'erro: {np.abs(I-2):.2e}')
```

```
integral aproximada: 1.99983550
erro: 1.64e-04
```

12. aproximação da derivada

[79]:
```
x = np.linspace(0, 1, 10**6)

f = x**3
df = np.gradient(f, x)

erro = np.max(np.abs(df - 3*x**2))
print(f'erro máximo em [0,1]: {erro:.2e}')
```

```
erro máximo em [0,1]: 3.00e-06
```

13. broadcasting

[81]:
```
A = np.arange(1, 10).reshape(3, 3)
v = np.array([10, 20, 30])

print(A + v)
```

```
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

14. exercício adicional – SVD

[100]:
```
A = np.arange(1, 7).reshape(3, 2)

U, S, Vt = np.linalg.svd(A, full_matrices=False)

print(f'U = {U}')
print(f'diag(S) = {np.diag(S)}')
print(f'Vt = {Vt}')

erro = la.norm(A - U @ np.diag(S) @ Vt)

print(f'erro: {erro:.2e}')
```

```
U = [[-0.2298477   0.88346102]
 [-0.52474482  0.24078249]
 [-0.81964194 -0.40189603]]
```

```
diag(S) = [[9.52551809 0.        ]
 [0.         0.51430058]]
Vt = [[-0.61962948 -0.78489445]
 [-0.78489445  0.61962948]]
erro: 1.39e-15
```

observa que a decomposição SVD possui erro $\mathcal{O}(10^{-15})$, que é pequeno, mas não é nulo.